

Lista de Prioridades

Estruturas de Dados e Algoritmos I

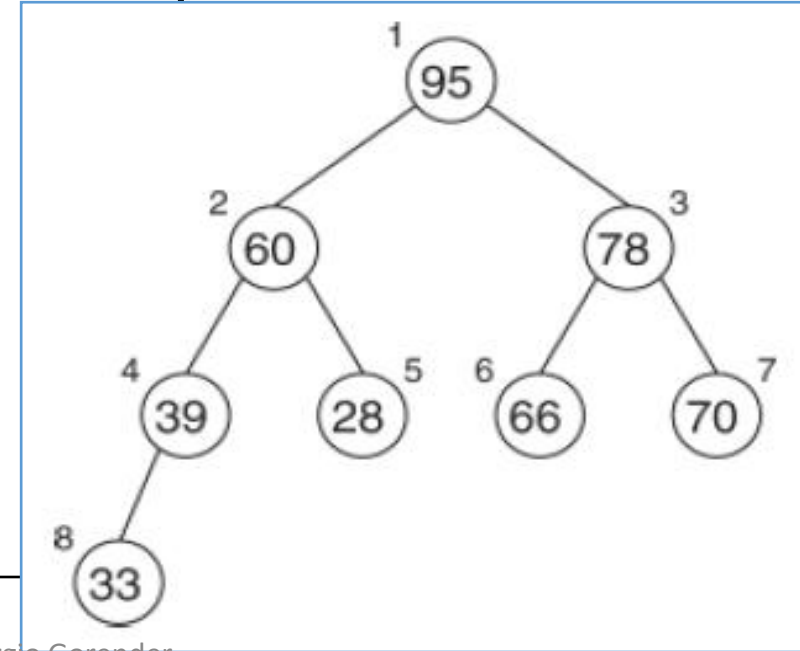
Prof. Sérgio Gorender

- Uma lista de prioridades pode ser definida como uma tabela na qual cada um de seus dados está associado a uma prioridade. Essa prioridade é, em geral, definida através de um valor numérico e armazenada em algum de seus campos.
- As operações básicas a serem efetuadas com os dados da lista de prioridades são:
 - Seleção do elemento de maior prioridade;
 - Inserção de um novo elemento;
 - Remoção do elemento de maior prioridade.

- Implementação por lista não ordenada:
 - Operações de remoção e alteração na lista são custosas, envolvendo percorrer toda a lista.
 - Seleção: $O(n)$
 - Inserção (no final): $O(1)$
 - Remoção: $O(n)$
 - Alteração: $O(n)$
 - Construção: $O(n)$

- Implementação por lista ordenada:
 - Operações de inserção e alteração na lista são custosas, envolvendo percorrer toda a lista. A remoção passa a ser imediata.
 - Seleção: $O(1)$
 - Inserção (percorre a lista): $O(n)$
 - Remoção: $O(1)$
 - Alteração: $O(n)$
 - Construção: $O(n \log n)$

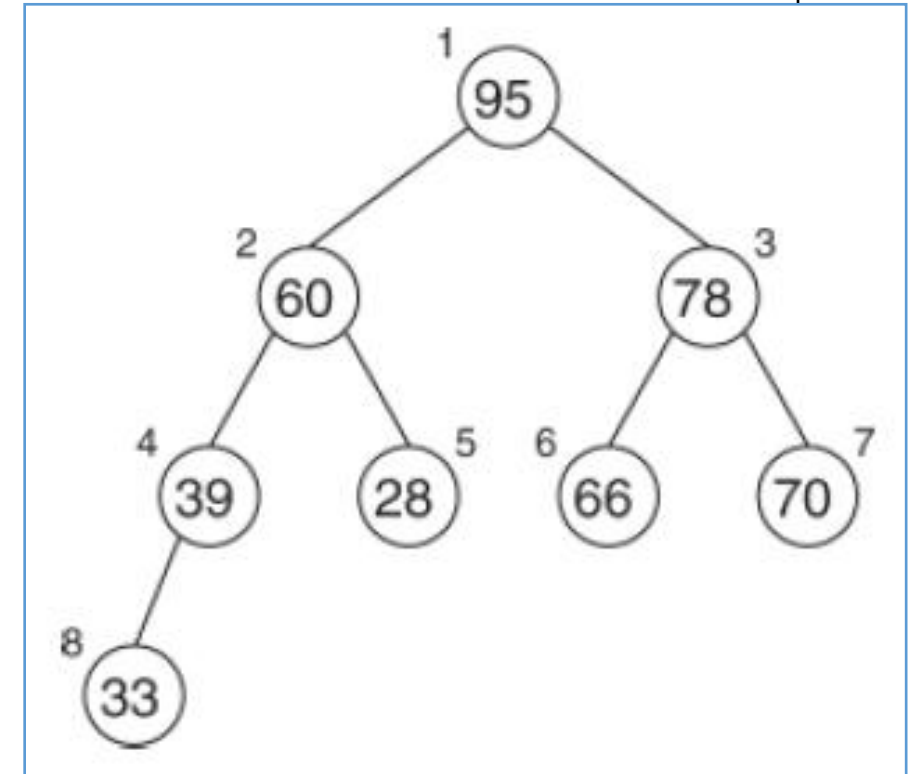
- Implementação por *heap*:
- Um *heap* é uma lista linear composta de elementos com chaves $s_1, \dots, s_n$, tais que $s_i \leq s_{\lfloor i/2 \rfloor}$, por exemplo, a chave do elemento **2** será maior que as chaves dos elementos **4** e **5** pois a divisão inteira de **4** e de **5** por **2** dá como resultado o valor **2**.
- Ex. **95 60 78 39 28 66 70 33**
- Esta lista é melhor representada graficamente por uma árvore binária completa.
- *Heap* de máximo – para todo nó i diferente da raiz, o valor do nó pai é maior ou igual ao do nó -
 $T[\text{pai}(i)] \geq T[i]$



Exercícios

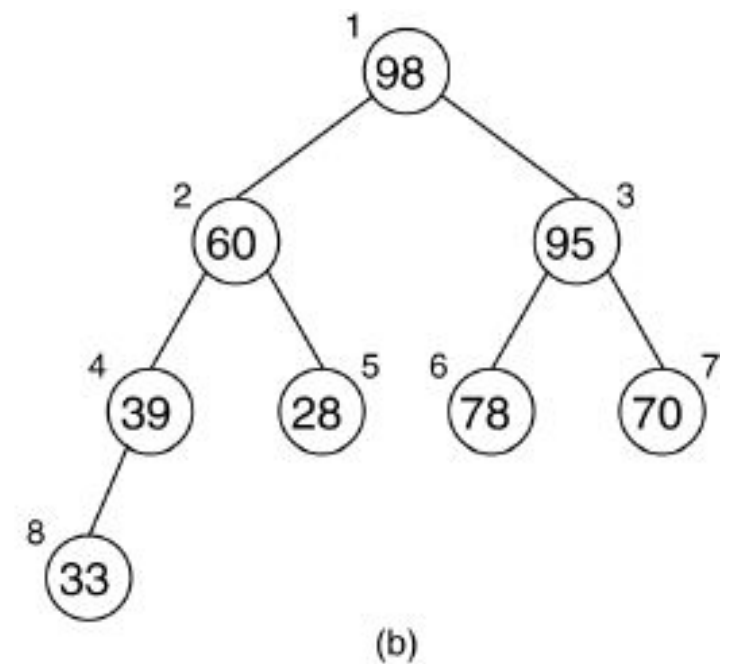
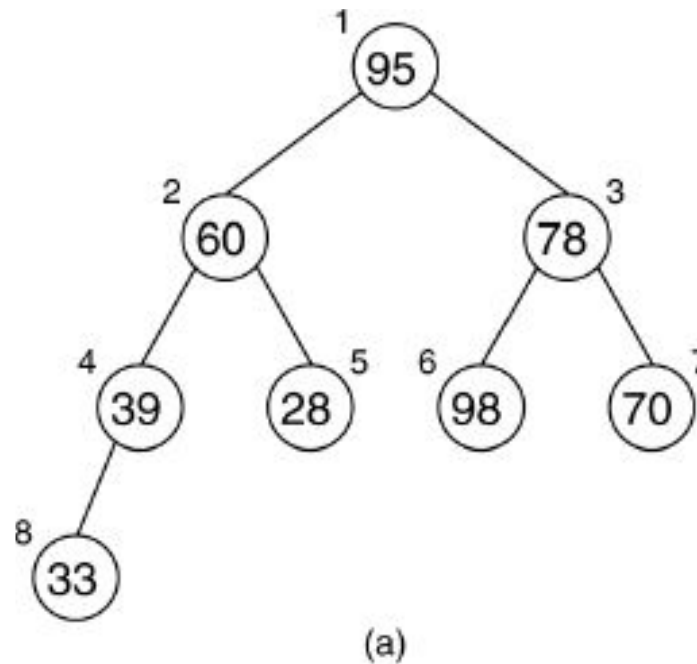
- Criar um heap de máximo com a seguinte lista de valores – 18, 25, 41, 34, 14, 10, 52, 50, 48.

- Implementação por *heap*:
- O elemento de maior prioridade será sempre o primeiro da ordenação, ou seja, a raiz da árvore.
 - Seleção: $O(1)$
 - Inserção: $O(\log n)$
 - Remoção: $O(\log n)$
 - Alteração: $O(\log n)$
 - Construção: $O(n)$



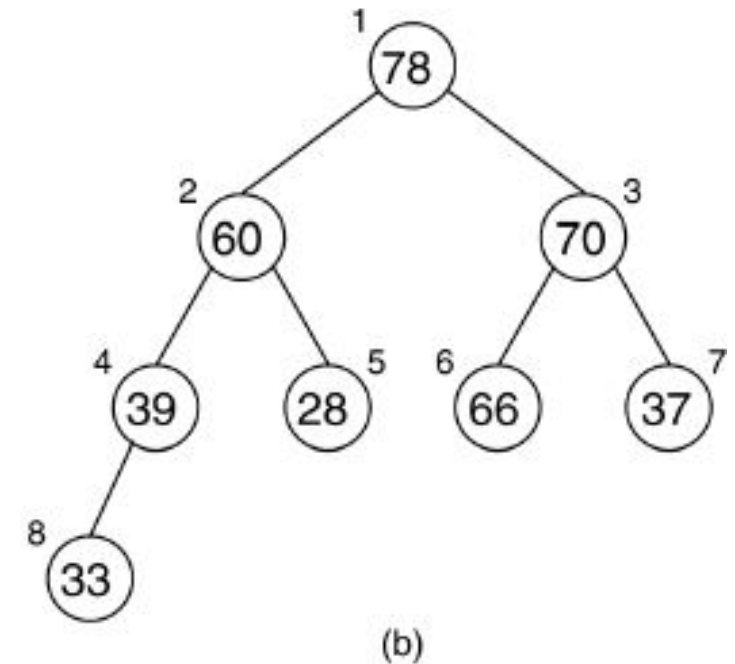
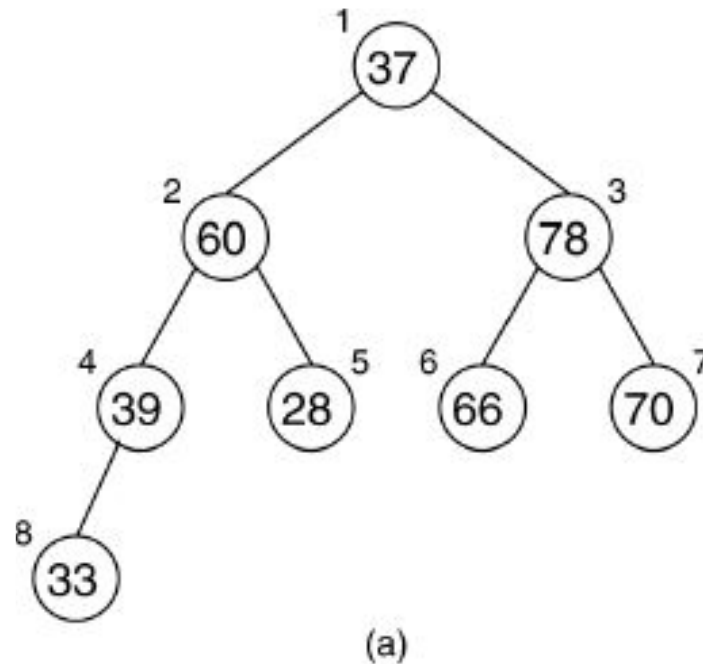
Alteração de prioridade

- Alterar a prioridade de um nó é caracterizado por subir ou descer este nó em um caminho da árvore.
- No exemplo, tendo alterado a prioridade do nó 6 para 98, este precisa subir na árvore para respeitar as propriedades da lista de prioridades.



Alteração de prioridade

- Neste exemplo, reduzimos a prioridade do nó 1 para 37, e neste caso, este nó precisa descer na árvore, por um ramo.



Algoritmo Subir

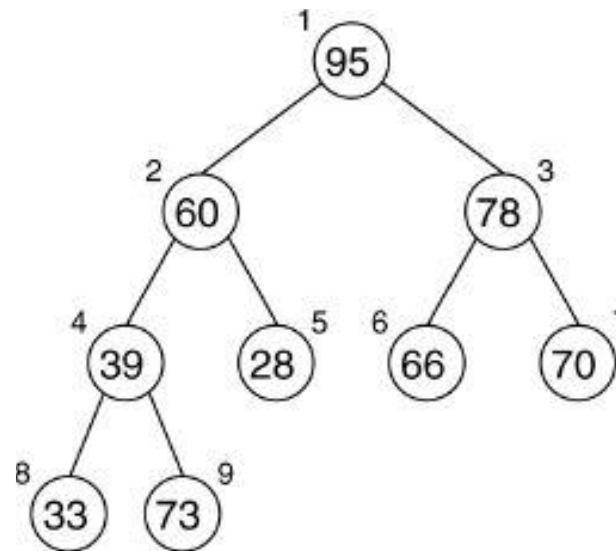
```
function subir(i)  
     $j = \lfloor i/2 \rfloor$   
    if  $j \geq 1$  then  
        if  $T[i].chave > T[j].chave$  then  
             $T[i] \Leftrightarrow T[j]$   
            subir(j)
```

Algoritmo Descer

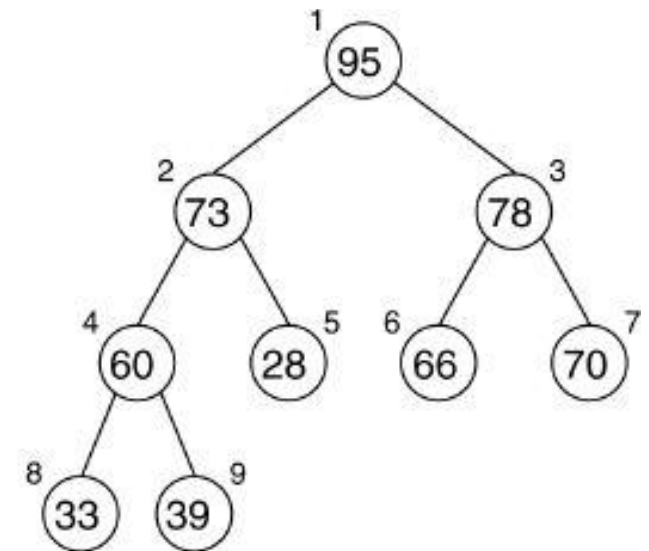
```
function descer(i, n)  
     $j = 2 * i$   
    if  $j \leq n$  then  
        if  $j < n$  then  
            if  $T[j+1].chave > T[j].chave$  then  
                 $j = j + 1$   
        if  $T[i].chave < T[j].chave$  then  
             $T[i] \Leftrightarrow T[j]$   
            descer (j, n)
```

Inserção

- Novo elemento, 73 é inserido no final da lista.
- Depois sobe pelo ramo em direção à raiz até a sua posição adequada.



(a)



(b)

Inserção

- Novos elementos são inseridos na lista de prioridades na primeira posição vazia, ou seja, $n+1$.
- Após a inserção é feita a adequação da prioridade deste elemento, subindo em um ramo em direção à raiz, até a sua posição adequada.

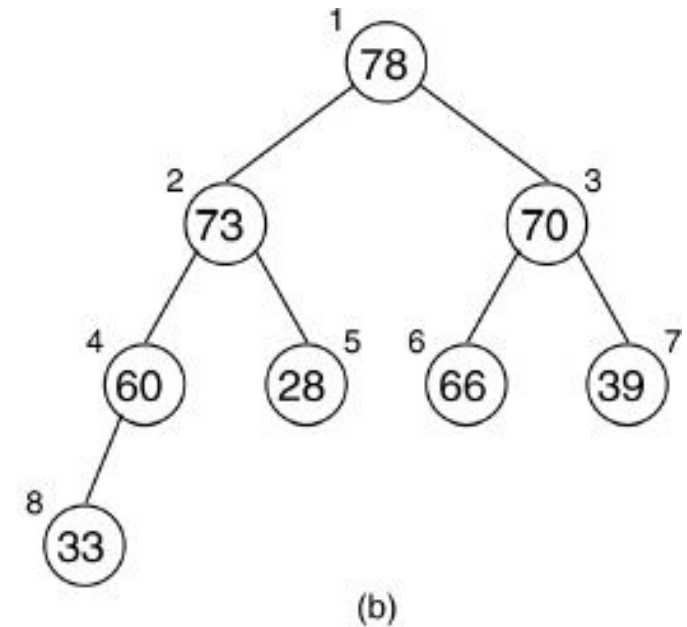
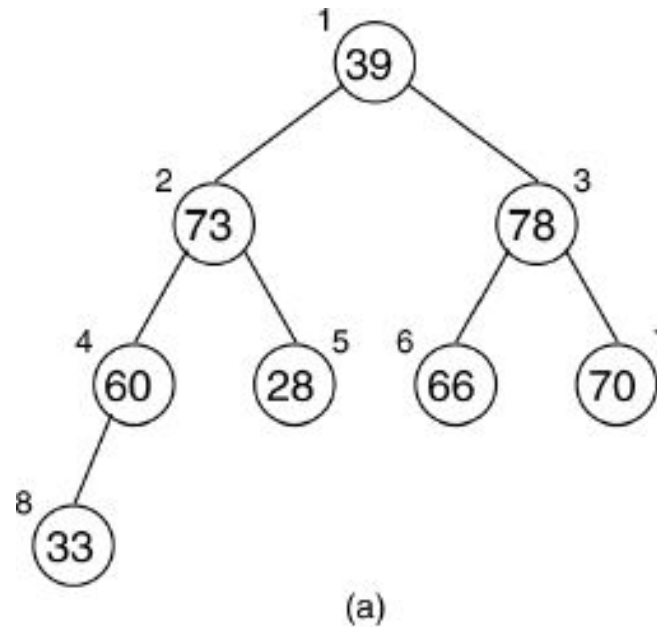
Algoritmo Inserção

```
if  $n < M$  then  
     $T[n+1] = novo$   
     $n = n + 1$   
    subir( $n$ )  
else overflow
```

Complexidade – $O(\log n)$

Remoção

- Removendo o elemento de maior prioridade (raiz da árvore), esta posição deverá ser substituída pelo elemento de menor prioridade (último da lista), reduzindo a prioridade do primeiro elemento. Este deverá então descer em um ramo da lista.
- No exemplo remove o 95, sendo substituído pelo 39.



Remoção

Algoritmo Remoção

if $n \neq 0$ **then**

$agir(T[1])$

$T[1] = T[n]$

$n = n - 1$

$descer(1, n)$

else *underflow*

Complexidade $O(\log n)$

- Aplicação - ordenação - HeapSort
- Para ordenar uma lista em ordem crescente:
 - Constrói a lista de prioridades, assumindo cada chave como sendo a prioridade do nó.
 - Troca o 1o elemento pelo último, decrementa o tamanho da tabela em 1, e desce o 1o elemento.
 - Repete este procedimento até o 2o elemento da tabela.

```
Algoritmo:  
arranjar(T, n)  
m = n  
while m > 1  
    T[1] <-> T[m]  
    m = m - 1  
    descer(1,m)
```

Exemplo: 40 37 95 42 23 51 27

